cuDESeq2: GPU acceleration of the DESeq2 differential-expression pipeline

Max Highsmith [1,*]

[1]Synthesize Bio

[*]To whom correspondence should be addressed: max@synthesize.bio

Intended article type: *Original Paper*. Subject: Gene expression.

Abstract

Motivation: DESeq2 is widely used for bulk RNA-seq differential-expression analysis. Its dispersion and negative-binomial GLM fits are performed gene by gene and can be slow when many analyses are needed.

Results: cuDESeq2 implements the standard DESeq2 Wald workflow in PyTorch and batches the gene-wise calculations on a CUDA GPU. Against R DESeq2 1.52.0, size factors agreed to floating-point precision and median relative dispersion error was 8.5e-4 across six real designs (7–300 samples; P = 2–5). All 76/76 step-parity tests on synthetic fixtures passed. The workflow includes count-outlier replacement and refitting. On an A100, end-to-end runtime was 10.3–99.2× lower than single-threaded R.

Availability and implementation: `gpu-deseq` requires PyTorch and a CUDA GPU, and is open source under the MIT license at `https://github.com/synthesizebio/gpu_deseq`, with reproduction scripts in `bench/` and `benchmarks/`.

Contact: max@synthesize.bio

Supplementary information: Appended to this manuscript.

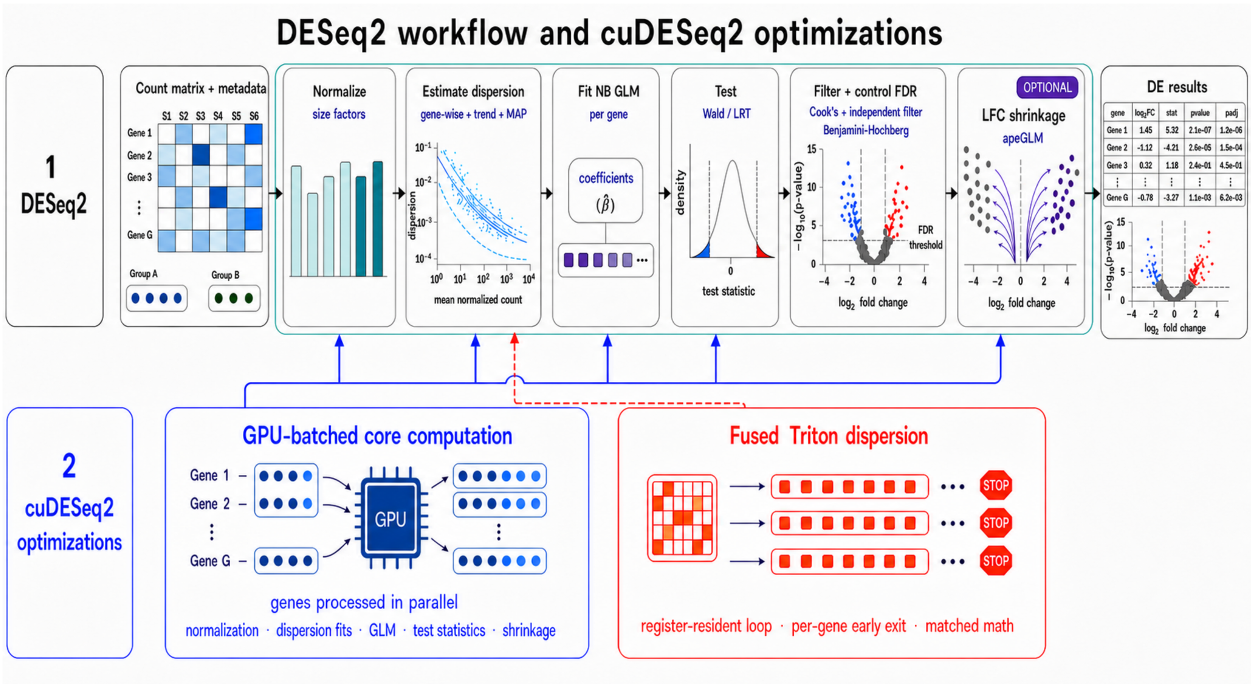

Figure 1: cuDESeq2 implements the five stages of a DESeq2 analysis. The gene-wise dispersion, GLM, and apeGLM calculations are batched on the GPU. Dispersion fitting can use eager PyTorch, CUDA-graph replay, or a fused Triton kernel (Table 1).

1 Introduction

DESeq2 (Love et al., 2014) fits a negative-binomial model to bulk RNA-seq counts. A Wald analysis estimates size factors and dispersions, fits the model, tests coefficients, and can shrink log fold changes with apeGLM (Zhu et al., 2019). Dispersion fitting, GLM fitting, and shrinkage are iterative calculations that can be parallelized across genes. The R package runs them on the CPU.

This cost matters when differential expression is evaluated repeatedly, for example in benchmarks of gene-expression and virtual-cell models (Bunne et al., 2024; Roohani et al., 2025). Some such benchmarks use rank tests on normalized expression instead (Roohani et al., 2025; Wei et al., 2026).

glmGamPoi (Ahlmann-Eltze and Huber, 2020) provides a faster CPU Gamma–Poisson fit and can be used by DESeq2. PyDESeq2 (Muzellec et al., 2023) reimplements DESeq2 in Python, with results that are similar but not identical to the R package. GPU single-cell packages such as ScaleSC accelerate PCA, graph construction, and clustering (Hu et al., 2025); they do not target bulk negative-binomial inference.

cuDESeq2 is a PyTorch implementation of this inference path. It follows R DESeq2 1.52.0 at intermediate steps and offers eager, CUDA-graph (NVIDIA Corporation, 2024), and Triton (Tillet et al., 2019) dispersion paths. The largest real-data p95 relative dispersion error was 4.4×10^{-3} (Sec. 3.2).

2 Methods

2.1 The DESeq2 pipeline and its cost profile

DESeq2 (Love et al., 2014) models each gene independently with a negative-binomial GLM. For gene g with counts K_{gi} over samples $i = 1, \ldots, S$, design matrix $X \in \mathbb{R}^{S\times P}$, and size factors s_i, it assumes $K_{gi} \sim \mathrm{NB}(\mu_{gi}, \alpha_g)$ with $\mu_{gi} = s_i \exp(x_i^\top \beta_g)$ and variance $\mu_{gi} + \alpha_g \mu_{gi}^2$, and runs five stages. We write S for the sample count and P for the number of model terms throughout; where a formula mirrors R's own source we keep R's names $m \equiv S$ and $p \equiv P$ for it, and dataset sizes are quoted as $n \equiv S$.

1. **Normalization**: median-of-ratios size factors.
2. **Dispersion estimation**: gene-wise Cox–Reid MLE, a parametric mean–dispersion trend, empirical-Bayes MAP shrinkage, and an outlier rule. This is the dominant cost on the largest design tested here.
3. **GLM fit / testing**: IRLS negative-binomial GLM, Wald / LRT.
4. **Significance**: Cook's-distance filter, independent filtering, Benjamini–Hochberg adjustment.
5. **LFC shrinkage**: apeGLM (Zhu et al., 2019), a Cauchy-prior MAP estimate of the effect size.

Stages 2, 3 and 5 contain per-gene iterative optimizations, and each is parallel across genes. On the largest design, dispersion estimation dominates the wall time (Sec. 3.3) because its gene-wise MLE maximizes the Cox–Reid adjusted profile log-likelihood

$$\ell_{\mathrm{CR}}(\alpha_g) = \sum_{i=1}^{S} \log \mathrm{NB}\big(K_{gi}; \mu_{gi}, \alpha_g\big) \; - \; \tfrac{1}{2} \log \det\big(X^\top W_g X\big), \tag{1}$$

with $W_g = \mathrm{diag}\big(\mu_{gi}/(1 + \alpha_g \mu_{gi})\big)$. The $\log\det$ Cox–Reid adjustment couples every sample and has to be recomputed at each iteration.

2.2 GPU implementation

We reproduce the reference computation step by step so that intermediate quantities agree with R (Sec. 2.8). In the eager tensor path, each per-gene loop in R becomes a program over all G genes at once: the per-gene solver state ($\log \alpha_g$, β_g, step sizes, iteration counters, convergence flags) is carried as $(G, \cdot)$ tensors, and a gene that has converged is frozen by a mask so that its later updates are no-ops. Batch speeds are constrained to the slowest gene's iteration count. Genes that are zero in every sample are dropped before fitting and their outputs padded with NaN, as in R.

Three of the five stages need no adjustment beyond this batching. *Normalization* computes median-of-ratios size factors from a 0.5 quantile rather than a median primitive, so that the averaging of the two middle values at even sample counts matches R; when every gene carries at least one zero it falls back to the positive-counts geometric mean. The *GLM fit* is a batched IRLS negative-binomial solve: working weights $W = \mu/(1 + \alpha\mu)$, working response $z = \log(\mu/s) + (K - \mu)/\mu$, and normal equations $(X^\top W X + \lambda I)\beta = X^\top W z$ solved for all genes in one batched factorization (ridge $\lambda = 10^{-6}$, deviance-ratio convergence $< 10^{-8}$, at most 250 iterations; a gene passing $|\beta| = 30$ is marked diverged). It returns β, an unthresholded μ, which Cook's distances need raw, and the hat-matrix diagonals; the few genes that reach the cap or diverge are re-fit one at a time by L-BFGS-B on the host, mirroring DESeq2's `optim`-based fallback for the same genes. Wald standard errors follow from the sandwich form $\mathrm{SE} = \sqrt{c^\top (M+\lambda I)^{-1} M (M+\lambda I)^{-1} c}$, with $M = X^\top \tilde{W} X$ built from a μ floored at 0.5 to match `fitBeta.cpp`. Without that floor a sample with $\mu < 0.5$ contributes almost no weight instead of a small one, M effectively loses it, and the SE inflates on degenerate genes (Sec. 3.2.1). The LRT statistic is the full/reduced deviance difference.

Significance contains no per-gene optimizer; Cook's distances and the BH adjustment are closed-form. A gene is flagged when its largest Cook's distance over samples in design cells of at least three replicates exceeds $F_{0.99,P,S-P}$, computed against a trimmed method-of-moments dispersion floored at 0.04, and fewer than three samples carry more counts than the sample holding that maximum. Independent filtering sweeps 50 base-mean quantile cutoffs, runs BH at each, and selects the first whose rejection count comes within one RMS residual of the peak of a robust LOWESS fit to the rejection curve.

The public `deseq()` entry point runs the standard Wald path: normalization, dispersion estimation, the Wald fit, Cook's-distance replacement and refitting for eligible design cells, and result assembly. The lower-level functions remain available for stage-wise validation and for LRT analyses. We use "drop-in" in this computational sense: the standard input model and reported DE quantities are reproduced. It does not imply that the Python API accepts every R object or every optional DESeq2 estimator (Sec. 4.1).

2.3 Batched dispersion estimation

The dispersion stage is a batched port of DESeq2's `fitDisp` and the `estimateDispersions` logic around it. Two inputs must be built the way R builds them, because the fit's accept-and-revert behavior depends on where the optimizer starts: the initial dispersion is $\mathrm{clip}\big(\min(\alpha_{\mathrm{rough}}, \alpha_{\mathrm{MoM}}), 10^{-8}, 10\big)$ from a linear-regression and a method-of-moments estimate, and $\hat{\mu}$ comes from plain linear regression for saturated designs and one IRLS pass at the initial dispersion otherwise. The stage has three parts.

(i) Gene-wise Cox–Reid MLE. We maximize Eq. 1 over $a = \log \alpha$ by gradient ascent with Armijo backtracking. Each iteration proposes $a' = a + \kappa\,\partial_a \ell_{\mathrm{CR}}$, shrinking κ as needed to keep a' inside $[-30, 10]$, accepts when the Armijo condition holds ($\varepsilon = 10^{-4}$), and adapts the step: on acceptance $\kappa \leftarrow \min(1.1\kappa, \kappa_0)$ with $\kappa_0 = 1$ and a halving every fifth acceptance, on rejection $\kappa \leftarrow \kappa/2$. A gene stops once an accepted step improves the objective by less than 10^{-6} or a falls below $\log(\mathrm{minDisp}/10)$, capped at 100 iterations. Two R-matching safeguards follow: a *noIncrease revert* restores the initial estimate for genes whose objective never improved on it, and a *grid fallback* re-fits the genes R would flag as non-converged (those at the cap or with a single accepted step), whose dispersion still exceeds $10 \cdot \mathrm{minDisp}$, using a deterministic 100-point coarse grid over $\log \alpha$ refined by a second. Estimates are clipped to $[10^{-8}, \max(10, S)]$: R's ceiling is $\max(10, S)$, and a fixed 10 truncates legitimate dispersions above ten samples.

(ii) Parametric trend. We fit $\alpha \approx a_0 + a_1/\bar{\mu}$ over the genes with dispGeneEst $>$ $100 \cdot \mathrm{minDisp}$ from $(a_0, a_1) = (0.1, 1)$. Each iteration forms residuals $\alpha/(a_0 + a_1/\bar{\mu})$, keeps the genes whose residual lies in $(10^{-4}, 15)$, and re-fits a Gamma-family, identity-link GLM warm-started at the current coefficients, stopping when $\sum \log(\mathrm{coef}/\mathrm{coef}_{\mathrm{old}})^2 < 10^{-6}$ with the inner GLM converged, or after ten iterations; a trimmed-mean trend is the fallback. R's other two `fitType` settings are also implemented: `"local"`, a LOWESS of $\log \alpha$ on $\log \bar{\mu}$ at span 0.3 with three robustness iterations, and `"mean"`, which forces the trimmed-mean trend.

(iii) MAP shrinkage. A log-normal prior centered on the trend shrinks the gene-wise estimates. Its variance follows R's three regimes, each built from the MAD s of the log-residuals: the closed form $\max\big(s^2 - \psi_1(\frac{m-p}{2}),\, 0.25\big)$ at ample residual degrees of freedom, a stochastic KL-divergence grid search when $0 < m - p \leq 3$, and s^2 itself for a saturated design. The MAP step re-runs the same gradient ascent from $\log \mathrm{dispGeneEst}$ with the prior term enabled and, matching R, with the revert and the grid fallback disabled. An outlier rule then keeps the raw MLE wherever $\log \alpha_{\mathrm{MLE}}$ exceeds the trend by more than $2s$.

The KL-divergence branch is the only stochastic part of the pipeline. It is used when

$0 < m - p \leq 3$ to estimate the dispersion-prior variance. We reproduce R's random-number stream and its `loess` interpolation because substitutions changed the estimate. On `airway` (m=8, p=5), the implementation returns the same prior variance as R. The random-stream replay is sequential and adds about 0.3 s per dataset. Implementation details and the validation of this branch are provided in the supplementary algorithms and tests.

2.4 Three execution modes for the dispersion loop

The gradient-ascent loop is where the launch-bound overhead concentrates (Sec. 3.3), so it is the only stage with alternative execution backends. The three modes compute the same mathematics and differ in how the loop is dispatched (Table 1). They are selected per call (`fit_dispersions(..., use_cuda_graph, use_triton)`) or through the `GPU_DESEQ_ACCEL` environment variable.

Table 1: The three execution modes for the dispersion loop.

Mode	Dispatch	Fidelity	Designs
eager	one kernel per tensor op	reference	any P
graph	replay a captured CUDA graph	bit-identical to eager	any P
Triton	one fused kernel per gene	R parity, not bit-identical	$P \in \{2, \ldots, 6\}$

Both accelerated modes fall back to eager on CPU; Triton also falls back for $P > 6$. Triton's agreement with eager is quantified in Sec. 3.2.

2.5 CUDA-graph chunked replay

The graph backend captures ten optimizer iterations and replays the captured chunk until convergence. This avoids a full 100-iteration capture while reducing launch overhead. Genes that converge within a chunk remain in the graph as no-op updates until the next convergence check.

CUDA graphs cannot capture the required cuSOLVER calls. We therefore use a no-pivot LU routine for the Cox–Reid determinant and trace terms in both eager and graph modes. The graph cache is keyed by matrix shape, chunk size, prior state, dtype, and device. The graph and eager paths are bit-identical.

2.6 Triton full-loop fusion

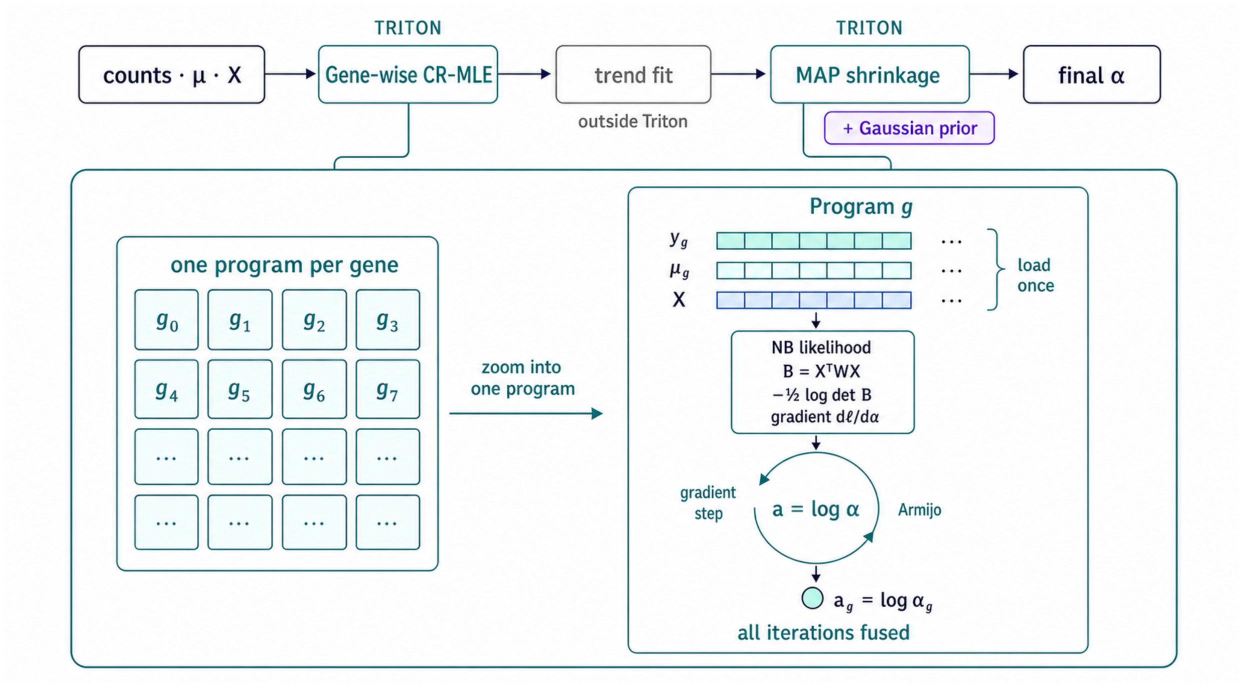

Figure 2: Fused Triton dispersion fitting. One program is launched per gene and runs the Cox–Reid optimizer in registers. The trend fit remains outside the kernel. Supplementary Algorithms S1–S2 give pseudocode.

The Triton backend launches one fp64 program per gene (Fig. 2). The program evaluates the likelihood, Cox–Reid term, gradient, and Armijo line search in registers, and exits when that gene converges. It supports $P \in \{2, \ldots, 6\}$; wider designs use the eager backend. The same fallback and revert rules are applied after the gene-wise fit.

The fused reduction order and the Triton `digamma` implementation change the final bits, so Triton is not bit-identical to eager. At fixed α, its log-posterior and gradient agree with eager to 3e-15 and 3e-14 relative, respectively. The real-data comparison is reported in Sec. 3.2.

2.7 Batched port of the apeGLM optimizer

apeGLM (Zhu et al., 2019) shrinks the log-fold-change by maximizing a per-gene posterior: the NB likelihood times a heavy-tailed Cauchy prior on the coefficient of interest and a wide Gaussian (SD 15, the DESeq2 default) on the nuisance coefficients. The Cauchy scale is set by empirical Bayes, $\tau = \min(\sqrt{\widehat{\text{var}}}, 1)$, with $\widehat{\text{var}}$ the root of apeGLM's prior-variance equation in the MLE coefficients and their Wald standard errors, solved once per dataset on the host.

For extreme-effect, low-count genes, the apeGLM posterior can have two nearly equal modes. R reports the solution reached by its initialized L-BFGS optimizer, not necessarily the global optimum. cuDESeq2 therefore uses the same optimizer family, initialization, stopping criteria, and iteration limit, vectorized across genes on the GPU. This matches the reference shrinkage behavior more closely than a different optimizer.

As in `lfcShrink`, only the shrunken coefficient and its standard error are replaced; p-values remain those from the Wald fit.

2.8 Parity methodology

Reference fixtures are generated by `scripts/generate_r_fixtures.R` against R DESeq2 1.52.0: three single-factor designs at 12×200, 30×500 and 60×2000 (samples $\times$ genes, $P = 2$), a 60×1500 two-factor design $\sim\text{batch} + \text{condition}$ with a three-level batch ($P = 4$), and a 60×1000 continuous-covariate design ($P = 2$). Every intermediate is checked against R: size factors, `dispGeneEst`, `dispFit` (the parametric trend on every fixture, plus the `"local"` and `"mean"` trends on the single-factor ones), `dispMAP`, the final dispersion and its outlier set, β, μ, the hat-matrix diagonals, Cook's distances, Wald SE, the LRT statistic on all five designs, the independent-filtering `padj`, and the apeGLM prior scale and shrunk LFC/SE. Errors are reported as p95 relative values, or absolute values where a quantity crosses zero. Sec. 3.2 reports the measured agreement for the principal quantities and tabulates the five pipeline stages on the real datasets (Table 2).

3 Results

3.1 Experimental setup

All GPU measurements are taken on one NVIDIA A100-SXM4-40GB under PyTorch 2.6 with CUDA 12.4 and NVIDIA driver 550.90.07. The reference CPU baseline (host A) is R DESeq2 1.52.0 on a 12-core box, with R 4.6.0 and apeglm 1.34.0. Every timing is a warm median of N repetitions with `torch.cuda.synchronize()` on both sides of the timed region, so no GPU work is left in flight when the clock stops. R runs single-threaded in the primary comparisons.

We validate on three real, published RNA-seq datasets sliced into six designs spanning $P = 2$–5 model terms: pasilla (Brooks et al., 2011) (*Drosophila*, n=7, one- and two-factor), airway (Himes et al., 2014) (human, n=8, three design slices), and a 300-sample GTEx (GTEx Consortium, 2020) whole-blood vs. skeletal-muscle contrast (54,922 genes). The first two are distributed as Bioconductor experiment packages; the GTEx counts are recount2's uniform reprocessing of SRA study SRP012682 (Collado-Torres et al., 2017), from which we draw 150 samples per tissue at a fixed seed and convert base coverage to read counts by dividing by the average read length. Scaling studies use fixed-seed negative-binomial simulations where the gene and sample axes can be swept independently.

3.2 Numerical parity with R

Correctness is checked per substep against R on all six real datasets (Table 2). A separate synthetic step-parity suite (`tests/test_r_step_parity.py`, 76/76) confirms the same agreement at controlled sizes, and more tightly: size factors agree to <1e-10, gene-wise dispersion to ∼1e-9, and MAP dispersion, β, μ, the Hessian and Wald SEs to ∼1e-6 relative. A further suite (`tests/test_triton_dispersion.py`) pins the fused kernel against the eager path once per covered P, the axis along which its specialized arms (Sec. 2.6) could diverge without any other test noticing.

Table 2: Per-substep agreement with R DESeq2 1.52.0 across all six real datasets. Each dataset is scored independently by the substep's natural metric (n=6; the score is per-dataset, not a pooled statistic), and both the median and the least-favorable of the six are shown. The R reference here is the substep chain (`estimateSizeFactors` → `estimateDispersions` → `DESeq` → `results`). Because size factors and dispersions already exist, the `DESeq` call times the Wald fit and, where applicable, count-outlier replacement and refitting. Reproduce: `bench/bench.py` → `bench/results/reference_parity.json`.

Substep	Metric	Median	Worst
normalization	max rel. Δ	4.1e-15	5.7e-15
dispersion	p95 rel. Δ	8.5e-4	4.4e-3
glm_fit	p95 $\|\Delta\|$ (LFC)	2.4e-5	8.1e-5
significance	Jaccard@0.05	1.00000	0.99963
lfc_shrink	Pearson r	0.99999	0.99722

All 30 real-data substep checks passed. The largest p95 dispersion difference was 4.4e-3 (0.44%) on `gtex_blood_muscle`. apeGLM shrinkage had Pearson $r \geq 0.997$ on every dataset. Graph mode was bit-identical to eager in 25 synthetic dispersion configurations; Triton differed only in the final bits ($\leq$3.2e-7 on the five small-n sets). The GTEx comparison includes count replacement and refitting in both implementations.

3.2.1 Parity in the standard DE plots

Figure 3 compares dispersion, MA, and volcano plots using the same plotting code and R-derived axis limits. The 141 dispersion outliers and 3,993 genes with padj < 0.05 are the same in both outputs.

Across all six designs, the top-N lists shared at least $\geq$99.6% of genes for every $N \leq 2{,}000$ (Fig. 4). Called-set Jaccard indices were at least $\geq$0.961 over $\alpha \in [10^{-3}, 0.2]$; four designs matched exactly at $\alpha = 0.05$.

Matching R required a μ floor when computing Wald standard errors. Without it, standard errors were 22% high for genes with an all-zero contrasted group. Applying the floor reduced the error on those genes to 0.03% and restored exact called sets for `airway_cell` and `airway`.

The standard wrapper replaces Cook's-distance outliers and refits eligible genes when a

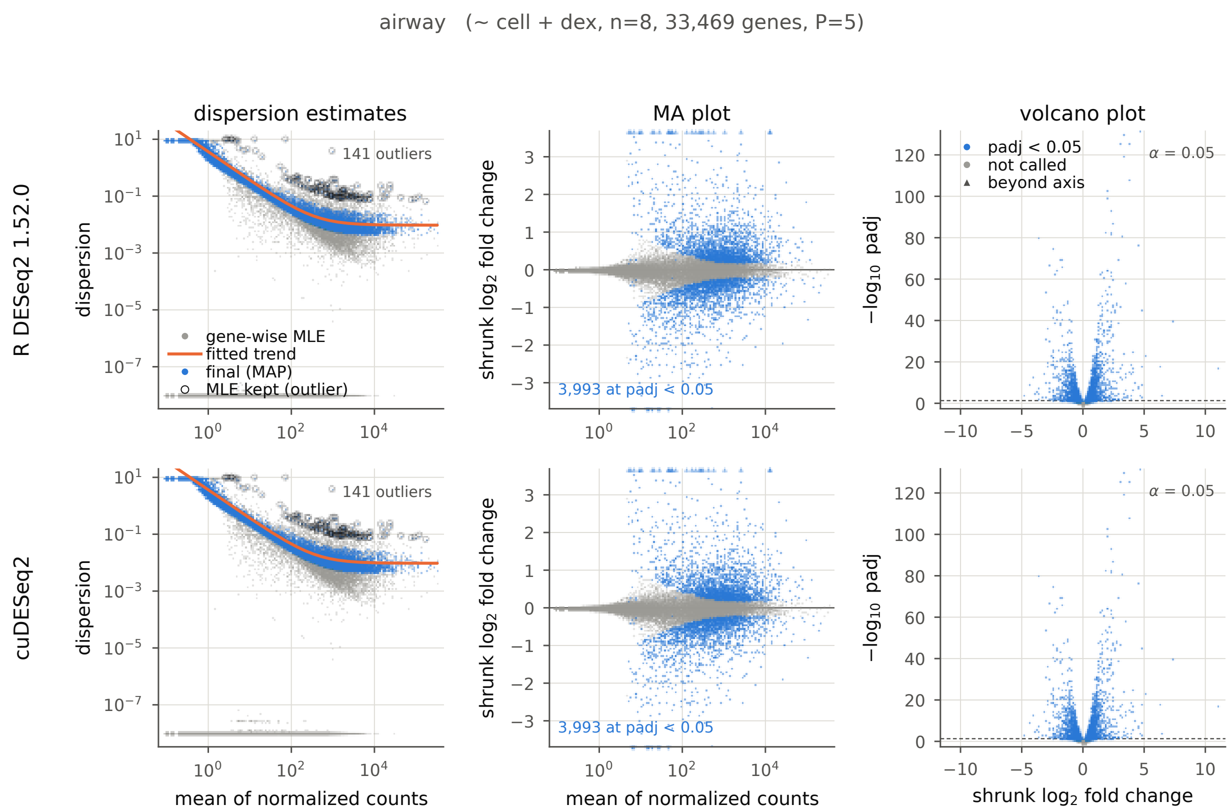

Figure 3: DESeq2 diagnostics for `airway` ($\sim$cell + dex, n=8, P=5): R DESeq2 1.52.0 (top) and cuDESeq2 (bottom). Left, dispersion estimates; centre, MA plot of apeGLM-shrunk LFC; right, volcano plot. The same code and axis limits were used for both rows. Reproduce: `validation/export_r_dispersion_details.R` then `validation/make_paper_figures.py`.

design cell has at least seven samples. This branch is exercised by GTEx (150 samples per group) and is included in both pipelines.

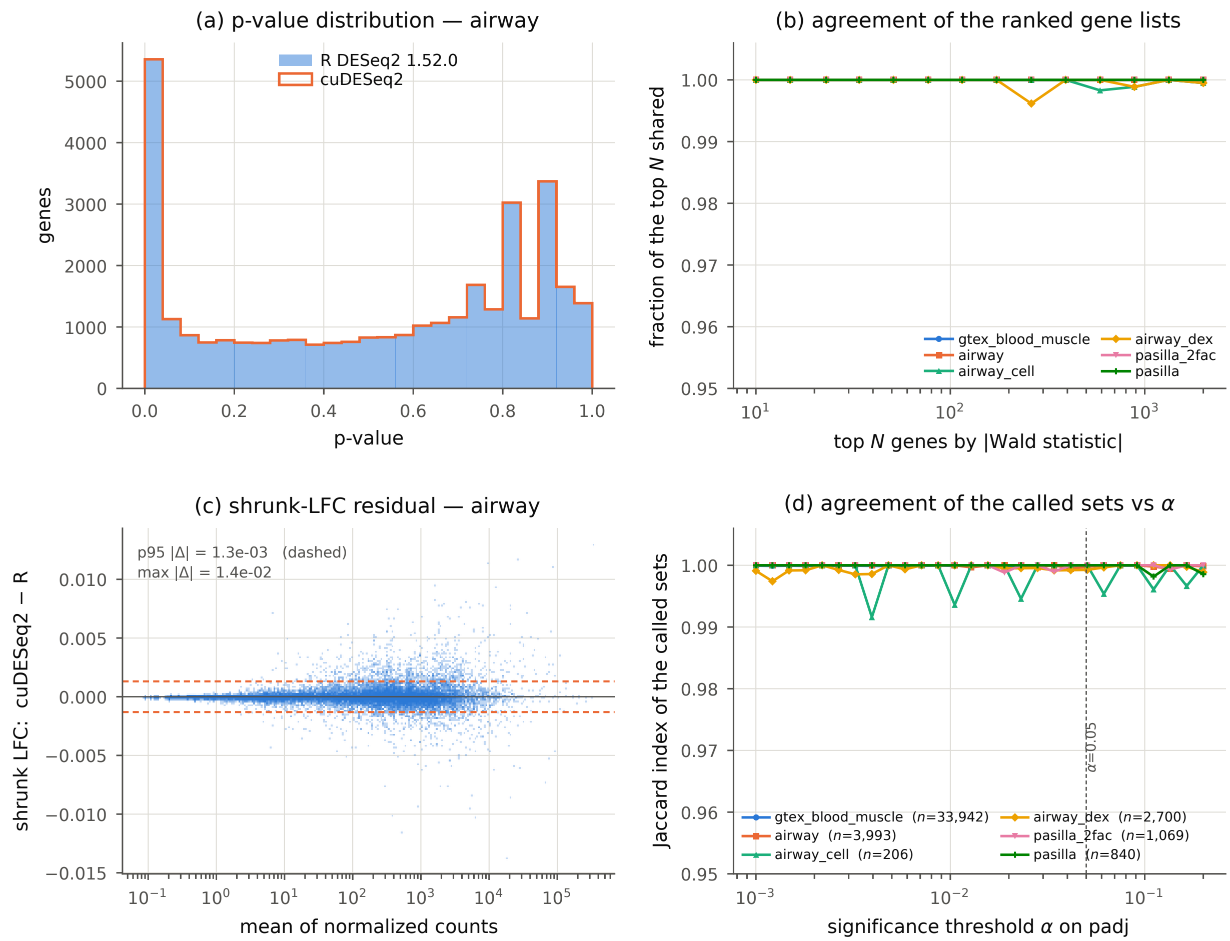

Figure 4: Quantitative agreement across all six designs. *(a)* p-value distributions on `airway`, overlaid. *(b)* Fraction of the top N genes shared by the two rankings, ranked by |Wald statistic| to avoid comparing tie-breaks among genes whose p-value has underflowed in R. *(c)* apeGLM-shrunk LFC residual against expression, showing no mean-dependent bias; dashed lines mark p95 $|\Delta|$. *(d)* Jaccard index of the called sets as the significance threshold moves, with the number of genes R calls at α=0.05 in the legend. The index is granular to $\sim 1/n$, so the small called sets move in visible steps. The `gtex` curve now includes count replacement and refitting in both pipelines.

3.3 Real-data timing versus R

Only the dispersion backend differs between modes; outlier refits use the same selected backend. Triton gave the shortest total time on five designs, while graph was fastest on

Table 3: End-to-end wall time on the real datasets: single-threaded R 4.6.0/DESeq2 1.52.0 and cuDESeq2 on one A100. All six GPU rows are from the same standard-pipeline benchmark session; the `gtex` row includes count replacement and affected-gene refitting. Reproduce: `bench/run_r.R` and `bench/bench.py` → `bench/results/timings.json`.

			R DESeq2 (s)	cuDESeq2, one A100 (ms)		
Dataset	P	n	1 thread	eager	graph	Triton
pasilla	2	7	11.1	982	650	717
pasilla_2fac	3	7	12.1	1481	922	841
airway_dex	2	8	30.9	1307	968	898
airway_cell	4	8	36.1	4169	3579	3495
airway	5	8	41.6	2559	1681	1504
gtex	2	300	471.6	8510	8293	4754

The best-mode speedup over single-threaded R is 10.3–99.2×; `gtex` is the upper endpoint.

`pasilla`. Triton was fastest for dispersion on all six designs. Graph supports any P and is bit-identical to eager; Triton is limited to $P \leq 6$ and differs in the final bits.

3.4 Scaling on synthetic data

Synthetic data separates the gene and sample axes. Hardware counters were not available, so we used an empirical roofline. A fp64 streaming probe sustained 1369 GB/s (88% of the A100 specification), while the dispersion kernels used 3–43% of that bandwidth. The objective-and-gradient evaluation took 1292 µs at 2,000 genes and 1287 µs at 20,000 genes. This weak dependence on gene count is consistent with launch and dispatch overhead dominating the loop.

The synthetic scaling experiments below isolate the dispersion optimizer rather than report end-to-end DESeq2 speedups; they therefore do not exercise the standard replacement/refit wrapper branch. The real-data timings in Table 3 use that branch where applicable. If the loop is launch-bound, the accelerators should pay most where iterations are many and each is cheap, and fade as every iteration acquires enough arithmetic to fill the device. The sample axis tests that directly (Table 5). Holding genes at 2,000 and sweeping S from a 2v2 pilot to a 2,000-sample cohort, the graph's speedup on the isolated gene-wise ascent

Table 4: Per-substep wall time (ms) on the real datasets, single-threaded R on the reference host vs. the three cuDESeq2 modes. The `glm_fit` timing includes Cook's-distance replacement and refitting. Reproduce: `bench/bench.py` → `bench/results/TABLES.md`.

Dataset	Substep	R	eager	graph	Triton
pasilla	dispersion	1722	402	71	20
	glm_fit	3878	38	37	37
	significance	206	63	63	63
	lfc_shrink	5143	478	478	596
	total	**11091**	**982**	**650**	**717**
pasilla_2fac	dispersion	2140	708	161	80
	glm_fit	3755	320	321	322
	significance	185	74	60	60
	lfc_shrink	5912	378	379	378
	total	**12137**	**1481**	**922**	**841**
airway_dex	dispersion	4707	429	109	51
	glm_fit	9924	55	53	53
	significance	1397	145	124	123
	lfc_shrink	14715	676	681	670
	total	**30932**	**1307**	**968**	**898**
airway_cell	dispersion	5794	748	186	60
	glm_fit	9584	37	36	36
	significance	362	114	115	115
	lfc_shrink	20157	3268	3240	3283
	total	**36080**	**4169**	**3579**	**3495**
airway	dispersion	8077	1586	713	581
	glm_fit	12686	62	60	60
	significance	872	117	117	117
	lfc_shrink	19712	793	790	746
	total	**41570**	**2559**	**1681**	**1504**
gtex	dispersion	188957	3500	3623	200
	glm_fit	239691	3147	2810	2709
	significance	496	623	623	621
	lfc_shrink	39330	1235	1233	1220
	total	**471588**	**8510**	**8293**	**4754**

`normalization` omitted ($\leq$4 ms cuDESeq2 on every set); full versioned R records are in `bench/results/timings.json`. All GPU values come from the same A100 session.

loop declines from 7.6× at S=4 to 0.96× at S=2000: by two thousand samples the launch overhead it removes is a rounding error against the per-iteration work, and replay is no longer worth its bookkeeping. Triton decays over the same range but never loses, from 69.0× to 7.7×, since fusion removes the intermediate traffic as well as the launches. Two features of the sweep are structural rather than noise. The S=4 and S=6 rows run the full 100-iteration cap while every larger S converges in 13–25, which is most of why the small-sample regime is both the slowest and the most accelerable. And S=4 is the one row where the stage-level speedup (1.8×) falls far below the loop-level one (69.0×): with $m - p = 2$ that design takes the stochastic KL branch (Sec. 2.3), whose compiled replay of R's random stream is serial host work no GPU mode touches, contributing ≈470 ms to every mode at S=4 and nothing at S=6. It is Amdahl's remainder for that row.

Table 5: Sample-axis scaling at 2,000 genes, `~condition` (P=2), single A100. "GW loop" is the isolated gene-wise Cox–Reid gradient-ascent fit (Sec. 2.3), the part the accelerators act on; "full stage" is `fit_dispersions` around it, including host-side trend fitting. Speedups are against eager in the same row. The graph is bit-identical to eager at every S ($\max|\Delta| = 0$); Triton is not, and agrees to ≤3.0e-8 here. Reproduce: `benchmarks/bench_cuda_graph.py --sample-sweep` → `benchmarks/results_a100_samplesweep.json`.

		GW loop (ms)			GW speedup		full-stage speedup	
S	iters	eager	graph	Triton	graph	Triton	graph	Triton
4	100	350.2	46.1	5.1	7.59	68.99	1.61	1.81
6	100	335.2	46.7	5.1	7.18	66.05	6.20	40.35
30	13	44.9	10.8	3.1	4.15	14.29	3.81	13.70
60	15	51.5	11.4	3.2	4.51	16.11	3.50	14.10
200	16	55.3	14.7	3.2	3.77	17.43	3.10	13.92
1000	18	63.9	44.0	8.0	1.45	8.02	1.30	8.17
2000	25	128.3	133.5	16.6	0.96	7.71	1.08	6.88

The gene axis says the same thing from the other side. Taking 2,000 to 20,000 genes at fixed S=60 multiplies the work tenfold but the eager stage time only by 1.7× at P=2 (127→210 ms) and 1.2× at P=4 (193→224 ms): nine tenths of the added genes are absorbed into launch latency the small case already paid. Because that fixed overhead is amortized over more genes at 20,000, it is also where the graph has least left to remove, its stage speedup dropping from 4.4× to 1.7× at P=2 (3.7× to 2.2× at P=4). Graph capture is a warm-path optimization: the one-time capture costs 487–796 ms across these four configurations, so it is recovered only when many matrices of one shape are fitted in a session, as in the serving pattern the virtual-cell evaluation loops of the Introduction generate.

3.5 Ablations

This section examines chunk length, per-gene early exit, and the apeGLM optimizer.

Chunk length. A full-length capture performed poorly in this benchmark. Because a captured graph cannot branch on a convergence flag, a capture of k iterations must be replayed $\lceil \text{iters}/k \rceil$ times, so the loop executes $\lceil \text{iters}/k \rceil \cdot k$ iterations rather than the iters eager early-exits after; done genes are frozen no-ops but still cost their share of the kernel. Sweeping k over the divisors of maxit = 100 (Table 6) traces that cost model. The full-length capture k=100 rounds 26 real iterations up to 100 and gives back almost all the benefit: 1.83× instead of 6.01× at 2,000 genes, 0.99× instead of 2.42× at 20,000, where it is indistinguishable from not graphing at all. What the sweep shows is that the padded iteration count, not the replay count, sets the time: rows executing the same number of iterations land within a couple of per cent of each other however many replays that took (k=25 and k=50 both pad to 50 and differ by 0.4% at 2,000 genes; k=2 through k=20 all pad to 40 and differ by 2% at 20,000). At 2,000 genes k=2 is fastest (6.01×) because it avoids padding, whereas at 20,000 genes k=10 and k=20 tie at 2.42×. The current k=10 default is therefore a conservative compromise rather than a universal optimum.

Per-gene early exit in the fused kernel. The Triton kernel's loop is guarded per program by `(it < maxit) & (~done)` (Sec. 2.6) rather than run for a fixed trip count, and the iteration counts of Table 5 bound what that is worth: outside the small-sample regime the loop converges in 13–25 iterations against a cap of 100, so a fixed-trip kernel would execute four to nearly eight times the loop body for an identical answer. The saving is largest exactly where the batch is widest, since a fixed trip count makes every gene wait

Table 6: `GRAPH_CHUNK` sweep on the isolated gene-wise loop, single A100, `~condition`. "iters run" is $\lceil \text{iters}/k \rceil \cdot k$, the padded count the replay actually executes; the eager loop early-exits after 26 and 40 iterations respectively. k=100 captures the whole loop in one graph and is therefore the fixed-full-length-capture ablation. Every k is bit-identical to eager ($\max|\Delta| = 0$), so these are pure scheduling trade-offs. Reproduce: `benchmarks/bench_cuda_graph.py --sweep` → `benchmarks/results_a100_sweep.json`.

	60 × 2,000 (eager 89.2 ms)			60 × 20,000 (eager 138.8 ms)		
k	iters run	graph (ms)	speedup	iters run	graph (ms)	speedup
2	26	14.83	**6.01**	40	58.24	2.38
5	30	16.28	5.48	40	57.48	2.41
10	30	16.18	5.51	40	57.29	**2.42**
20	40	20.73	4.30	40	57.45	**2.42**
25	50	25.42	3.51	50	71.15	1.95
50	50	25.34	3.52	50	70.96	1.96
100	100	48.73	1.83	100	139.66	0.99

for the cap rather than for its own convergence. This is a bound inferred from the shipped kernel's measured iteration counts rather than a measured A/B: the released code exposes no fixed-trip switch to compare against.

The apeGLM optimizer is a parity choice, not a speed choice. A batched Newton solver for the shrinkage step is both simpler to write and faster than porting apeGLM's L-BFGS, and it disagrees with R. The reason is the one described in Sec. 2.7: the Cauchy-prior posterior is bimodal for extreme-effect genes, R reports whichever mode its own optimizer reaches from its own initialization, and a solver that finds the global optimum therefore returns a better estimate that is not the reference's. This is the one place where improving on the reference and reproducing it are in direct conflict, and a drop-in replacement has to choose reproduction. Porting the reference's optimizer restores agreement to $r \geq 0.997$ on shrunk log-fold-changes across all six designs (Table 2) while keeping the stage on the GPU with no per-gene host fallback.

4 Discussion

cuDESeq2 uses fp64 and follows the R convergence tolerances, clamps, and optimization steps. Across the six real datasets, size factors, dispersions, coefficients, Wald statistics, adjusted p-values, and shrunken effects agreed closely with R (Table 2). The observed end-to-end speedup on the A100 was 10.3–99.2× relative to single-threaded R.

At the tested sizes, dispersion fitting is limited largely by kernel-launch and dispatch overhead. CUDA graphs and Triton reduce that overhead. Their benefit falls as the per-iteration calculation becomes larger. The reported ratios are specific to the A100, the CPU baseline, and single-threaded R; they should not be treated as hardware-independent estimates.

Compatibility depends on implementation details as well as the statistical model. Examples include R's random-number stream and `loess` behavior for the dispersion prior, the μ floor in `fitBeta.cpp`, and apeGLM's optimizer initialization. Omitting these details changed outputs on difficult genes even when the underlying equations were the same.

4.1 Limitations

- Tested designs: single factor (up to four levels), additive multi-factor (up to P=5), continuous. Not yet: interaction designs. Parity is not tested below n=7, the smallest real design here; the synthetic fixtures start at 12 samples.

- Not implemented: `lfcShrink` types `normal/ashr`, `glmGamPoi`, VST/rlog.

- Triton path: $P \in \{2, \ldots, 6\}$ (falls back to eager otherwise); not bit-identical to eager; agreement is $\leq$3.2e-7 in dispersion on the five small-n sets, with a worst case of 2.6e-1 relative on one `gtex` gene.

- Benchmarks: a single GPU (one A100), so we cannot report how the three modes reorder on a smaller or newer device. Parity is measured against DESeq2 1.52.0.

The remaining work includes the `normal` and `ashr` shrinkage estimators, interaction designs, and Triton support beyond P=6. The independent-filtering and dispersion-trend calculations remain on the host. Multi-GPU execution has not been evaluated.

Funding

This work was supported by Synthesize Bio. It received no specific grant from any funding agency in the public, commercial or not-for-profit sectors.

Conflict of Interest

All authors are affiliated with Synthesize Bio, which developed and maintains the software described here and released it under the MIT license. The authors declare no other competing interests.

Data and software availability

Every dataset used here is public and previously published; this work generated no new sequencing data. pasilla (Brooks et al., 2011) and airway (Himes et al., 2014) are the Bioconductor experiment packages of the same names, installable with `BiocManager::install`. The GTEx cohort (GTEx Consortium, 2020) is taken from recount2 (Collado-Torres et al., 2017) as the gene-level summarized experiment for SRA study SRP012682, downloaded from the `recount-opendata` S3 bucket. The scripts `fetch_and_reference.R` and `prepare_gtex.R`, both under `validation/`, fetch each dataset, fix the contrast levels, and write the exact count matrices and R DESeq2 reference outputs the comparisons use, so all six designs are reconstructible from the two scripts.

Source code, tests, and benchmark harnesses are available at `https://github.com/synthesizebio/gpu_deseq`, released under the MIT license. The package is distributed as `gpu-deseq` and imported as `gpu_deseq`; it requires Python 3, PyTorch and a CUDA GPU, with Triton for the fused mode. A Docker image definition pins R 4.6.0, Bioconductor 3.23, DESeq2 1.52.0, apeglm 1.34.0, PyTorch 2.6.0/CUDA 12.4 and the LaTeX toolchain. The same image runs CPU reference generation and, through the NVIDIA container runtime, the GPU benchmark. Reproduce with:

```
make container-build
make container-check
make container-test
make container-r-reference
make container-gpu-benchmark
make container-paper
```

References

Constantin Ahlmann-Eltze and Wolfgang Huber. glmGamPoi: fitting gamma–poisson generalized linear models on single cell count data. *Bioinformatics*, 36(24):5701–5702, 2020. doi: 10.1093/bioinformatics/btaa1009.

Angela N. Brooks, Li Yang, Michael O. Duff, Kasper D. Hansen, Jung W. Park, Sandrine Dudoit, Steven E. Brenner, and Brenton R. Graveley. Conservation of an RNA regulatory map between Drosophila and mammals. *Genome Research*, 21(2):193–202, 2011. doi: 10.1101/gr.108662.110.

Charlotte Bunne, Yusuf Roohani, Yanay Rosen, et al. How to build the virtual cell with artificial intelligence: priorities and opportunities. *Cell*, 187(25):7045–7063, 2024. doi: 10.1016/j.cell.2024.11.015.

Leonardo Collado-Torres, Abhinav Nellore, Kai Kammers, et al. Reproducible RNA-seq analysis using recount2. *Nature Biotechnology*, 35(4):319–321, 2017. doi: 10.1038/nbt.3838.

GTEx Consortium. The GTEx consortium atlas of genetic regulatory effects across human tissues. *Science*, 369(6509):1318–1330, 2020. doi: 10.1126/science.aaz1776.

Blanca E. Himes, Xiaofeng Jiang, Peter Wagner, Ruoxi Hu, Qiyu Wang, et al. RNA-Seq transcriptome profiling identifies CRISPLD2 as a glucocorticoid responsive gene that modulates cytokine function in airway smooth muscle cells. *PLoS ONE*, 9(6):e99625, 2014. doi: 10.1371/journal.pone.0099625.

Wenxing Hu, Haotian Zhang, Yu H. Sun, Shaolong Cao, Jake Gagnon, Yuka Moroishi, Yirui Chen, Zhengyu Ouyang, and Baohong Zhang. ScaleSC: a superfast and scalable single-cell RNA-seq data analysis pipeline powered by GPU. *Bioinformatics Advances*, 5(1):vbaf167, 2025. doi: 10.1093/bioadv/vbaf167.

Michael I. Love, Wolfgang Huber, and Simon Anders. Moderated estimation of fold change and dispersion for rna-seq data with DESeq2. *Genome Biology*, 15(12):550, 2014. doi: 10.1186/s13059-014-0550-8.

Boris Muzellec, Maria Teleńczuk, Vincent Cabeli, and Mathieu Andreux. PyDESeq2: a python package for bulk RNA-seq differential expression analysis. *Bioinformatics*, 39(9): btad547, 2023. doi: 10.1093/bioinformatics/btad547.

NVIDIA Corporation. CUDA C++ programming guide: CUDA graphs. `https://docs.nvidia.com/cuda/cuda-c-programming-guide/`, 2024. Accessed July 2026.

Yusuf H. Roohani, Tony J. Hua, Po-Yuan Tung, et al. Virtual Cell Challenge: toward a Turing test for the virtual cell. *Cell*, 188(13):3370–3374, 2025. doi: 10.1016/j.cell.2025.06.008.

Philippe Tillet, H. T. Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proc. 3rd ACM SIGPLAN Int. Workshop on Machine Learning and Programming Languages (MAPL)*, pages 10–19, 2019. doi: 10.1145/3315508.3329973.

Zhiting Wei, Yiheng Wang, Yicheng Gao, et al. Benchmarking algorithms for generalizable single-cell perturbation response prediction. *Nature Methods*, 23(2):451–464, 2026. doi: 10.1038/s41592-025-02980-0.

Anqi Zhu, Joseph G. Ibrahim, and Michael I. Love. Heavy-tailed prior distributions for sequence count data: removing the noise and preserving large differences. *Bioinformatics*, 35(12):2084–2092, 2019. doi: 10.1093/bioinformatics/bty895.

Supplementary algorithms

Supplementary Algorithm S1 gives a high-level view of the fused Triton dispersion fit sketched in Fig. 2; Supplementary Algorithm S2 gives the full kernel, including the Armijo line search, the step-size adaptation, and the Cox–Reid objective/gradient routine shared by the CR–MLE and MAP passes.

Supplementary Algorithm S1 FUSED TRITON DISPERSION FIT (high-level)

Require: counts $\mathbf{Y} \in \mathbb{N}_0^{G\times S}$; fitted means $\boldsymbol{\mu} \in \mathbb{R}_+^{G\times S}$; design $\mathbf{X} \in \mathbb{R}^{S\times P}$; initial dispersions $\boldsymbol{\alpha}^{(0)}$; optional MAP prior.
Ensure: optimized log dispersions $\mathbf{a}$.

Launch one Triton program for every gene g in parallel.
    Load $\mathbf{y}_g$, $\boldsymbol{\mu}_g$, and $\mathbf{X}$ once into program-local state.
    Initialize $a \leftarrow \mathrm{clip}(\log \alpha_g^{(0)}, -30, 10)$ and step size $\kappa \leftarrow \kappa_0$.
    repeat inside the same fused program
        Compute the negative-binomial log-likelihood and Cox–Reid adjustment.
        If fitting MAP dispersion, add the Gaussian prior on $a = \log \alpha$.
        Compute the gradient $q \leftarrow \partial \ell / \partial a$.
        Propose the gradient-ascent step $\widetilde{a} \leftarrow a + \kappa q$.
        Accept or reject $\widetilde{a}$ using the Armijo condition; adapt κ.
    until converged or *maxit* is reached.
    Store the final a_g once in global memory.

Supplementary Algorithm S2 DETAILED FUSED TRITON DISPERSION KERNEL (CR–MLE or MAP)

Require: counts $\mathbf{Y} \in \mathbb{N}_0^{G\times S}$; fitted means $\boldsymbol{\mu} \in \mathbb{R}_+^{G\times S}$; design $\mathbf{X} \in \mathbb{R}^{S\times P}$, $P \in \{2,\dots,6\}$; initial dispersions $\boldsymbol{\alpha}^{(0)} \in \mathbb{R}_+^{G}$; compile-time $h \equiv$ HASPRIOR; prior means $\mathbf{m}$ and variance σ^2 when $h=1$; initial step bound κ_0.
Ensure: optimized log dispersions $\mathbf{a}$ and iteration counts $\mathbf{c}$.

parallel for Triton program $g \in \{0,\dots,G-1\}$ **do** (one program per gene)
  Load $\mathbf{y}_g$, $\boldsymbol{\mu}_g$, and $\mathbf{X}$ once into program-local state.
  Set $a \leftarrow \text{clip}\Big(\log \alpha_g^{(0)}, -30, 10\Big)$, $\kappa \leftarrow \kappa_0$, $n_{\text{acc}} \leftarrow 0$, and $c \leftarrow 0$.
  Set $(\ell, q) \leftarrow \text{ObjectiveGradient}(a, \mathbf{y}_g, \boldsymbol{\mu}_g, \mathbf{X}, h, m_g, \sigma^2)$.
  while $c < \mathit{maxit}$ and not converged **do**
    $c \leftarrow c+1$ and propose $\widetilde{a} \leftarrow a + \kappa q$.
    If $q \neq 0$ and $\widetilde{a} \notin [-30, 10]$, shorten κ so that $\widetilde{a}$ lies on the violated boundary.
    Set $(\widetilde{\ell}, \_) \leftarrow \text{ObjectiveGradient}(\widetilde{a}, \mathbf{y}_g, \boldsymbol{\mu}_g, \mathbf{X}, h, m_g, \sigma^2)$.
    Set $r \leftarrow [\, \widetilde{\ell} \geq \ell + \varepsilon\kappa q^2 \,]$. (Armijo acceptance flag)
    Select $a' \leftarrow r\, ?\, \widetilde{a} : a$, $\ell' \leftarrow r\, ?\, \widetilde{\ell} : \ell$, and $\Delta \leftarrow \ell' - \ell$.
    Set $(\_, q') \leftarrow \text{ObjectiveGradient}(a', \mathbf{y}_g, \boldsymbol{\mu}_g, \mathbf{X}, h, m_g, \sigma^2)$ and $q \leftarrow r\, ?\, q' : q$.
    Set $n'_{\text{acc}} \leftarrow n_{\text{acc}} + r$ and $\kappa_{\text{acc}} \leftarrow \min(1.1\kappa, \kappa_0)$.
    If r and $n'_{\text{acc}} \bmod 5 = 0$, set $\kappa_{\text{acc}} \leftarrow \kappa_{\text{acc}}/2$.
    Set $\kappa \leftarrow r\, ?\, \kappa_{\text{acc}} : \kappa/2$, $n_{\text{acc}} \leftarrow n'_{\text{acc}}$, $a \leftarrow a'$, and $\ell \leftarrow \ell'$.
    Converge if $r \wedge (\Delta < 10^{-6} \;\vee\; a < \log(\mathit{minDisp}/10))$.
  end while
  Store $a_g \leftarrow a$ and $c_g \leftarrow c$ in global memory.
end parallel for
function ObjectiveGradient$(a, \mathbf{y}, \boldsymbol{\mu}, \mathbf{X}, h, m, \sigma^2)$
  Set $\alpha \leftarrow e^a$, $\mathbf{W} \leftarrow \text{diag}(\boldsymbol{\mu}/(1+\alpha\boldsymbol{\mu}))$, and $\mathbf{B} \leftarrow \mathbf{X}^\mathsf{T}\mathbf{W}\mathbf{X}$.
  Compute $\ell \leftarrow \ell_{\text{NB}}(\mathbf{y}; \boldsymbol{\mu}, \alpha) - \frac{1}{2}\log\det(\mathbf{B})$.
  Compute $q \leftarrow \partial\ell/\partial a$, including $\text{tr}\big(\mathbf{B}^{-1}\partial\mathbf{B}/\partial\alpha\big)$ via an unrolled 2×2 formula ($P=2$) or $P\times P$ LU solve.
  if $h = 1$ **then** $\ell \leftarrow \ell - (a-m)^2/(2\sigma^2)$ and $q \leftarrow q - (a-m)/\sigma^2$. (MAP only)
  return (ℓ, q).
end function